

VITYARTHI

Project report

GAME: ROCK–PAPER–SCISSORS

A Python Console-Based Interactive Game

Student name	Sahil Mangesh Kalbande
Registration / Applicationn No.	26BAI10488
Programme	B.Tech– COMPUTER SCIENCE AND ENGINEERING[AI & ML]
Academic Year	2026-2027
Course / Subject	Introduction to problem solving
Faculty / Mentor	M.K.JAYANTHI

1.Introduction

Rock–Paper–Scissors is a simple two-player decision game in which each player chooses one of the following: Rock, Paper, or Scissors. The winner is determined by the following three rules: Paper defeats Rock, Rock defeats Scissors, and Scissors defeats Paper. This project implements the game as an interactive Python console application in which the user plays against a computer-controlled opponent.

The project demonstrates fundamental programming concepts such as variables, lists, input/output, conditional statements, loops, exception handling, random number generation, indexing, and basic program control flow. The application also allows the user to play multiple rounds without restarting the program.

2. Problem Statement

To design and implement a reliable console-based Rock–Paper–Scissors game in Python where the user selects one of three choices, the computer generates a random choice, the program calculates the winner according to predefined rules, and the user can choose whether to play another round.

Scope of the Project

- : Single-user interaction against a computer opponent.
- : Three valid game choices: Rock, Paper, and Scissors.
- : Input validation for numeric choices from 1 to 3.
- : Handling of non-numeric input using exception handling.
- : Random computer choice using Python's random module.
- : Repeated rounds with a Y/N replay option.

Target Users

The project is intended for beginner Python learners, students demonstrating programming fundamentals, and users who want a small interactive console game.

3.Functional Requirements

ID	Requirement	Description
FR-01	Display rules	Show the three winning rules before gameplay.
FR-02	User input	Accept 1, 2, or 3 for Rock, Paper, or Scissors.
FR-03	Validation	Reject invalid or non-numeric input and request input again
FR-04	Computer choice	Generate a random value from 1 to 3
FR-05	Winner calculation	Determine tie, user win, or computer win
FR-06	Result display	Show both choices and the round result.
FR-07	Replay	Allow the user to play another round or exit.

4. Non-functional Requirements

category	Requirement
Usability	Simple numbered menu and clear prompts
Reliability	invalid input should not terminate the program unexpectedly
Performance	Each round should execute immediately with negligible delay
Maintainability	Code should use clear variable names and logical sections
Portability	The program should run on a standard Python 3 environment.
Readability	Output and code should be understandable to beginner programmers

4.2 statement.md

The project statement describes the problem, scope, target users, and expected behavior of the Rock–Paper–Scissors console application. It can be maintained as a Markdown file in the GitHub repository.

4.3 Source code/ project files

- Python source file containing the complete game logic.
- README.md describing the project and execution steps. statement.md containing the -problem statement and scope.
- Project report PDF containing documentation, diagrams, testing, and results.
- Screenshots of successful program execution, where required.

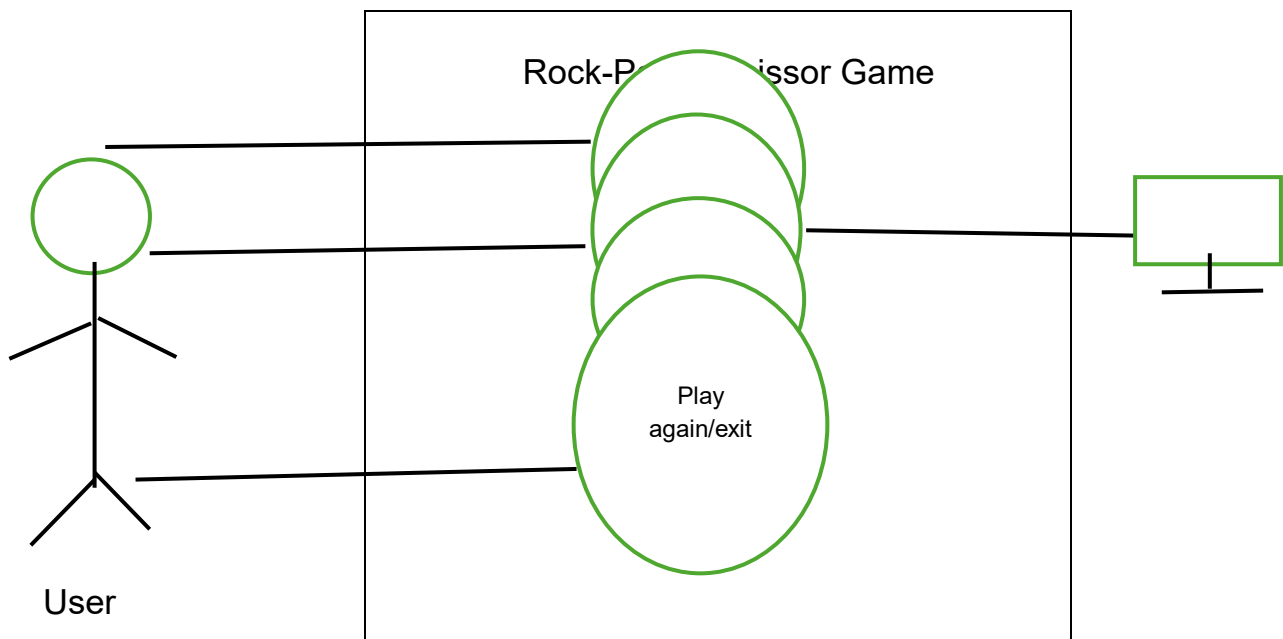
5. System Architecture

The application follows a simple console-based architecture. The user interacts with the Python program through the terminal. The program validates the input, uses the random module to generate the computer's choice, compares both choices, and displays the result.

Layer / Component Role

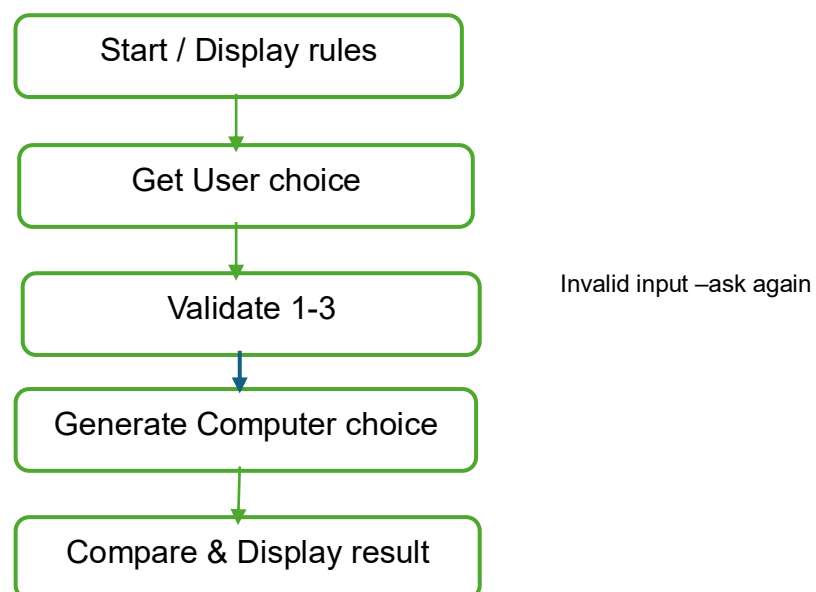
Layer / Component	Role
User Interface	Console prompts and printed results
Input Validation	Checks numeric and range validity
Game Logic	Compares user and computer choices.
Random Generator	Produces computer choice using <code>random.randint(1, 3)</code> .
Replay Controller	Repeats the game when the user enters Y

6.1 use case diagram



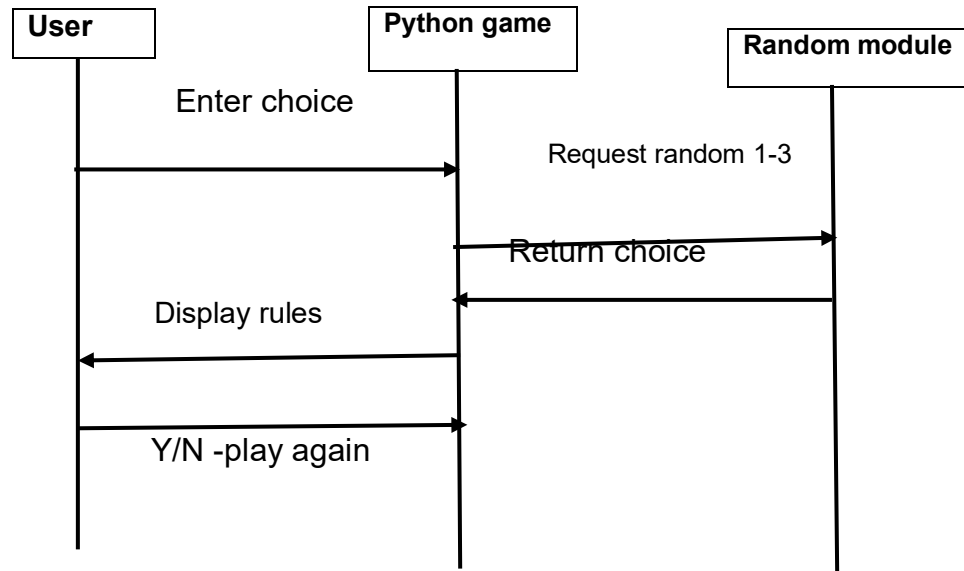
The user starts a game, enters a choice, plays a round, views the result, and may play again or exit. The computer opponent is represented by the program's random-choice mechanism.

6.2 Workflow Diagram



The workflow begins by displaying the rules and menu, then validates the user's choice, generates the computer choice, determines the result, and finally asks whether another round should be played.

6.3 Sequence Diagram



The sequence shows the interaction among the user, the Python game program, and the random-number generation module during one round.

6.4 Class / component diagram

Component	responsibility	Main elements
Game Controller	Controls overall game loop.	while True, replay loop
Input handler	Reads and validates user input.	input(), int(), try/except
Choice Manager	Maps numbers to choices.	choices list
Random Generator	Selects computer move.	random.randint(1, 3) if /
Winner Logic	Determines round outcome	if / elif / else

6.5 ER Diagram (if storage used)

No database or persistent storage is used in this version. Therefore, an ER diagram is not required. All game information exists only during the current program execution.

7.Design Deccisions & Rationale

Decision	Rationale
List for choices	Rationale A list makes it easy to map 1, 2, and 3 to the corresponding text choice
random.randint(1, 3).	Provides a simple selection among the three choices.
try/except ValueError	Prevents program termination when the user enters text instead of a number
While true	Supports continuous gameplay until the user chooses to exit
Integer comparison	Makes the winner rules concise and easy to understand
Y/N replay validation	Ensures the replay question receives an expected response

8.Implementation details

The program is implemented in Python 3. The random module is imported to generate the computer's choice. A list named choices stores the three possible moves. The user's number is converted into a list index by subtracting 1. The computer follows the same mapping.

Core Winner Logic

```

if choice == comp_choice
    print("<<< It's a Tie! >>>")
elif ((choice == 1 and comp_choice == 3) or
      (choice == 2 and comp_choice == 1) or
      (choice == 3 and comp_choice == 2)):
    print("<<< User wins! >>>")
else:
    print("<<< Computer wins! >>>")

```

Input Validation Logic

try:

```
choice = int(input("Enter your choice: "))
```

```
except ValueError:
```

```
    print("Please enter a valid number.")
```

```
    continue
```

```
while choice < 1 or choice > 3:
```

```
    choice = int(input("Please enter a valid choice (1-3): "))
```

9.Screenshots / Results

The supplied project screenshots show the Python source code for the Rock–Paper–Scissors game. The implemented program displays the rules, accepts a user choice, generates a computer choice, prints both choices, determines the result, and asks whether the user wants to play again.

Test scenario	Expected outp	
	u	
User and computer choose same move	It's a Tie	
Rock vs Scissors	User wins when user has Rock	
Paper vs Rock	User wins when user has Paper	
Scissors vs Paper	User wins when user has Scissors	
User enters 4	Program asks for a valid choice	
User enters text	Program displays a valid-number message	
User enters N at replay	Program exits with thanks message	

Sample Console Result

Welcome to Rock-Paper-Scissors!

Winning Rules:

Rock vs Paper -> Paper wins

Rock vs Scissors -> Rock wins

Paper vs Scissors -> Scissors wins

Choose an option:

1 - Rock

2 - Paper

3 - Scissors

Enter your choice: 1

User choice is: Rock

Now it's Computer's Turn...

Computer choice is: Scissors

Rock vs Scissors

<<< User wins! >>>

Do you want to play again? (Y/N): n

Thanks for playing!

10. Testing Approach

TC	Input	Expected result	Status
TC-01	1, computer=1	Tie	Pass
TC-02	1, computer=3	User wins	Pass
TC-03	2, computer=1	User wins	Pass
TC-04	3, computer=2	User wins	Pass
TC-05	1, computer=2	computer wins	Pass
TC-06	4	Ask for valid choice	Pass
TC-07	abc	Handle value error	Pass
TC-08	Y at replay	Start another round	Pass
TC-07	N at replay	Exit program	Pass

11. Challenges Faced

Handling invalid user input without crashing the program.

Making sure numeric choices correctly map to Rock, Paper, and Scissors.

Writing all winning combinations correctly.

Maintaining correct indentation required by Python.

Managing the replay loop so that the user can continue or exit.

Testing different combinations to confirm that the winner logic is correct.

These challenges were addressed through input validation, structured conditional logic, repeated testing, and careful use of Python indentation.

12. Learnings & Key Takeaways

Understanding Python variables, lists, and indexing.

Using if, elif, and else for decision making.

Using while loops for repeated execution.

Handling invalid input with try/except.

Using the random module for computer-generated choices.

Converting a real-world rule set into algorithmic conditions.

Testing a program with valid, invalid, and boundary inputs.

Understanding the importance of clean program structure and readable output.

13. Future Enhancements

Add a score counter for the user and computer.

Add a best-of-3 or best-of-5 game mode.

Add a graphical user interface using Tkinter or another GUI framework.

Add sound effects and animations.

Store match history in a file or database.

Add player names and a leaderboard.

Create a web version using HTML, CSS, JavaScript, or a Python web framework.

Add unit tests for the winner-determination function.

14. References

Python 3 documentation – built-in functions, control flow, and exception handling.

Python random module documentation – random integer generation.

Course / classroom notes on Python programming fundamentals.